

P2 Documentation and Code Project

You are viewing draft content
created with the use of Claude.AI



Getting Started with the Propeller 2

Meet the Chip, Read Its Code, Put It to Work

September 2026

Version 1.0.4 — Community Review Edition

Guide Organization

A short, friendly orientation to the Propeller 2 — meet the chip, learn to read its code, and put it to work.

The Three Chapters

- Ch 1 — Meet the Propeller 2
- Ch 2 — Reading P2 Code
- Ch 3 — Putting It to Work

Back Matter & Next

- Where to Next — the reference library
- Then: *The P2 Architect's Guide*

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Contents

Copyright and License	3
Acknowledgments	3
Sources	3
How to Use This Guide	4
Conventions	4
Chapter 1: Meet the Propeller 2	6
Eight Cogs — eight little computers	7
Memory — three tiers, from tiny-and-fast to big-and-shared	8
Pins and smart pins — I/O that thinks for itself	10
The CORDIC solver — shared math hardware	10
The streamer — moving data at full speed	11
Events and interrupts — noticing when something happens	11
The clock — one setting, the whole chip	12
Bootling — how a program starts running	13
Where this leaves us	13
Chapter 2: Reading P2 Code	14
A program is an object — six kinds of blocks	14
Methods — how the work is organized	15
Indentation is the structure	16
Values, names, and a line you'll see broken	16
Objects — building from other files	17
The other language: PASM2	17
Where this leaves you	18
Chapter 3: Putting It to Work	19
Your first program: drive a pin	19
Adding a second Cog	20
Sharing data between Cogs	21
Spin2 or PASM2? A decision, not a syntax tour	22
Objects and the run-time model	23
How it boots and runs	23
Where this leaves us	23
Where to Next	25

Copyright and License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution–ShareAlike 4.0 International License (CC BY-SA 4.0).

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose, even commercially

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made.
- **ShareAlike** — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

To view the full license, visit: <https://creativecommons.org/licenses/by-sa/4.0/>

Trademarks

Parallax, Propeller, Spin, and the Parallax logo are trademarks of Parallax Inc. This license grants permissions under copyright only; it does not grant rights to use these trademarks, and adapted or redistributed copies must not imply endorsement by, or official status with, Iron Sheep Productions, LLC or Parallax Inc.

Acknowledgments

This guide stands on work done by others:

Parallax Inc. for the Propeller 2 microcontroller and the reference documentation that defines its behavior.

Chip Gracey for designing the P2 — the eight-cog architecture, the smart pins, the CORDIC solver, and the streamer this guide teaches you to think with.

The P2 community whose drivers, projects, and hard-won design habits shaped how this guide frames “thinking in P2.”

Sources

This guide is a distillation, not a primary source. It draws on, and points you back to, these trusted P2 reference documents:

- **The Parallax Propeller 2 Documentation v35 - Rev B/C** (Chip Gracey, Parallax Inc.) — the architectural ground truth behind Chapter 1.
- **The Spin2 Reference Manual and the P2 Assembly Language Reference** — the language facts behind Chapter 2 (“Reading P2 Code”).
- **The P2 reference manuals** (Assembly Language, I/O & Smart Pins, Streamer, DEBUG) — the depth this orientation deliberately leaves to them (see *Where to Next*).

How to Use This Guide

This is a short, narrative guide, not a reference manual — it is meant to be *read*, and it is built so different readers can enter at different doors. Four paths:

- **New to the Propeller 2?** Read straight through: Chapter 1 builds the mental picture, Chapter 2 teaches you to *read* P2 code (so the examples ahead aren’t a mystery), and Chapter 3 puts it to work in real (compiling) code. Take the chapters in order; each earns the next.
- **Coming from the Propeller 1?** You already own the model — and Spin2 will look familiar. Skim Chapter 1 following the bronze **“P1 note”** sidebars — they call out exactly what’s *the same*, *changed*, or *new* on the P2 — skim Chapter 2 for what’s new in the language, then read Chapter 3 as a hands-on refresher.
- **Already writing P2 code?** Use this guide as a fast orientation and a reference for the chip and its code. When you’re ready to design a *whole system* on the P2 — deriving the right set of cooperating cogs and objects rather than an accidental sequential program on parallel silicon — that’s the companion volume, *The P2 Architect’s Guide*.
- **An AI agent or tool?** Read this guide for the narrative and the mental model — the way an experienced P2 developer thinks. The P2 reference manuals carry the exhaustive, authoritative facts the design decisions rest on.

Conventions

A few conventions run through the whole guide:

- **“cog,” never “CPU” or “core.”** The P2 community treats a cog as *the computer*, and so do we.
- **Code shows named constants, not raw numbers.** Examples use the compiler’s symbolic constants (a pin’s name, `_clkfreq`) the way you’d actually write them — and every code example compiles.
- **Code blocks are colored by language** — Spin2 in **blue**, PASM2 (assembly) in **green** — the same IDE-aligned scheme as the rest of the P2 manual family, so code is recognizable at a glance.
- **“P1 note” sidebars** (bronze boxes) are short asides for readers migrating from the Propeller 1, each labeled *same as P1*, *changed in P2*, or *new in P2*. A newcomer can skip every one of them without losing the thread.

- **Callouts are used sparingly** — the occasional **Tip** box flags a non-obvious orientation insight. This is a narrative guide, not a reference peppered with boxes.

Chapter 1: Meet the Propeller 2

By the end of this chapter you'll be able to picture the whole chip — eight processors, one shared pool of memory, and a ring of clever pins around the outside — and you'll know roughly what each part is for. That picture is all you need before we start writing code. We're not going to be exhaustive here; each part has a deep manual of its own, and we'll point you to it as we go. The goal right now is just to make the Propeller 2 feel familiar.

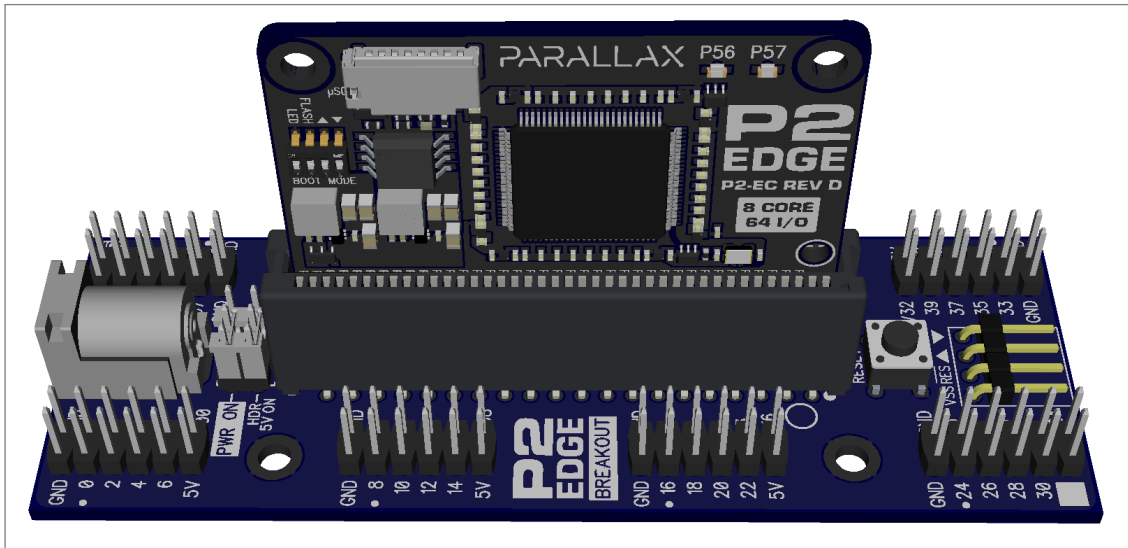
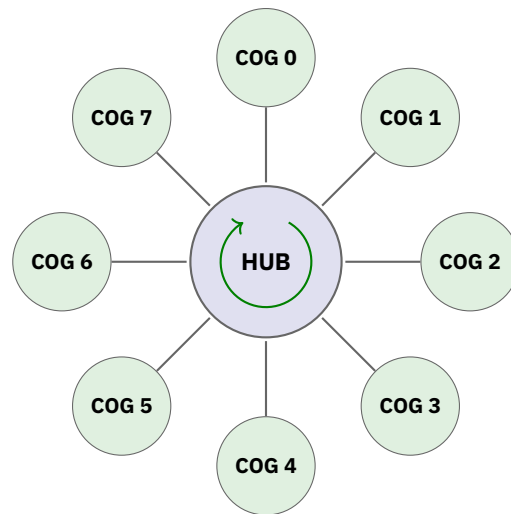


Figure 1. A P2 Edge module — the small board carrying the Propeller 2 chip — seated on a P2 Edge Breakout Board. This is the hardware the guide's examples assume. (Image courtesy of Parallax Inc.)

If you've worked with a microcontroller before, the most useful thing to know up front is where the P2 sits. A conventional microcontroller does one thing at a time, very fast: a single instruction stream racing down a single core, and when you need it to juggle several jobs at once, you lean on interrupts and a scheduler to slice that one core into time-shared pieces. At the other extreme, an FPGA is *all hardware all at once* — you describe logic and it becomes physical circuitry, with no instruction stream at all. The P2 lives in the gap between those two landmarks. It is still software — you write programs, the way you always have — but instead of one core to time-share, you get **eight** real processors that genuinely run at the same time, plus dozens of small hardware helpers around the edges that do their jobs on their own. You rarely have to choose between “fast single thread” and “dedicated hardware.” On the P2 you usually just hand each job its own piece of silicon.

That idea — *give each job its own piece of silicon, and let it keep running* — is the thread that runs through this whole guide. Let's meet the pieces.



All COGs access Hub simultaneously — Max 8 clocks wait
Each COG: Independent processor with 2KB RAM + 2KB LUT
Hub: 512KB shared memory accessible by all COGs

Figure 2. Eight independent cogs arranged around the shared hub, taking turns on a fixed rotation — the “egg beater.” (The 64 smart pins live on the outside of the chip; we meet them a few pages on.)

Eight Cogs — eight little computers

The heart of the P2 is its eight processors. Each one is called a **cog**, and the P2 community treats a cog as *the computer* — when a P2 programmer says “put that on its own cog,” they mean “give that job its own processor.” All eight are identical 32-bit processors, numbered 0 through 7, and they all run at the same time, at full speed, without getting in each other’s way.

This is the part that surprises people coming from a normal microcontroller. There is no scheduler handing out time slices, no context-switching, no operating system quietly deciding who runs next. Each cog just keeps running its own program, independently, from start to finish. One cog can sit in a tight control loop forever while another talks to a sensor and a third drives a display — and none of them slows the others down. Because nothing interrupts a cog from the outside, its timing is *deterministic*: the same code takes exactly the same number of clock cycles every time it runs. That predictability is why the P2 is so good at jobs where timing has to be exact, like generating video or driving a motor.

Each cog has a little private memory of its own to hold its program and data, a fast hardware path for calls and returns, and even its own dedicated streaming engine for moving data at high speed (we’ll get to that one shortly). You start a cog running with a single instruction and stop it with another; a stopped cog powers down until you need it again.

TIP

The mental shift that makes the P2 click is to stop thinking “one program that does everything” and start thinking “several small programs, each minding one job, running side by side.” Almost every P2 design is some version of that.

P1 NOTE

Same as P1: If you’re coming from the Propeller 1, this part will feel like home: the P2 keeps the same eight-cog, shared-hub family architecture that defined the original Propeller. Eight symmetric processors, no interrupts required, deterministic timing — all still true. What changed is *how much* each cog can do, which the rest of this chapter is about.

For the assembly-level execution details — the instruction pipeline, how a cog fetches and runs code, starting and stopping cogs from PASM2 — see Part I of the *P2 Assembly Language Reference*.

Memory — three tiers, from tiny-and-fast to big-and-shared

A cog works with memory at three levels, and it’s worth knowing them apart because the trade-off between them shapes a lot of P2 code.

Closest and fastest is each cog’s **private register RAM** — 512 longs (2 KB) that belong to that cog alone. It’s small on purpose: it sits right next to the processor, so access is immediate. Right beside it is a second private block, the **lookup RAM** (the “LUT”) — another 512 longs you can use for data, waveforms, or extra code. These two are private, quick, and limited.

Then there’s the **hub** — 512 KB of RAM shared by all eight cogs. This is the big common pool: where your larger programs live, where cogs leave data for each other, where buffers and tables sit. It’s far roomier than the private memories, with the trade-off that it’s shared, so reaching it involves a brief, predictable wait for your turn (more on that next).

Memory Hierarchy

Per-COG (Private)

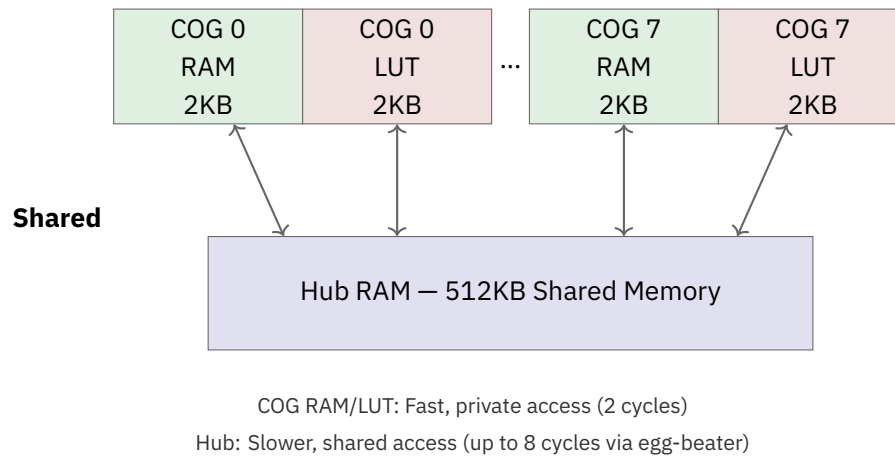


Figure 3. The memory tiers. Each cog’s private RAM and LUT sit right next to the processor for fast, near-immediate access; the 512 KB hub is shared by all eight (a few cycles’ wait via the egg beater).

P1 NOTE

Changed in P2: The shape is familiar — private cog RAM plus a shared hub — but the sizes are transformed. The P1 had 32 KB of hub; the P2 has **512 KB**. The 512-long cog register space is the same size you know, but the P2 *adds* the 512-long LUT alongside it, which the P1 didn’t have at all — and adjacent cogs can even share their LUTs for fast hand-offs.

How Cogs share the hub — the “egg beater”

Because all eight cogs share one hub, something has to decide who gets access when. On the P2 that “something” is a round-robin hardware mechanism nicknamed the **egg beater**, and the nice thing about it is that it’s completely predictable. Each cog is guaranteed its own access slot on a fixed rotation, so a hub read or write never fails and never stalls unpredictably — at worst you wait a few clocks for your slot to come around. And once you’re streaming a block of data, it flows at a rate of one long per clock. There’s no bus contention to reason about and no priority fights; the hardware takes turns, forever, on schedule.

This is the one place where a cog’s timing depends on the others, and even here it’s bounded and knowable rather than random — which is exactly what you want when you’re counting cycles.

P1 NOTE

Changed in P2: The P1 also shared its hub by strict rotation, so the take-turns idea is familiar. The P2's egg beater refines it: the rotation is tighter, and block transfers move a long every clock once you're synced, so the shared memory keeps up with high-speed work in a way the P1's hub couldn't.

Memory addressing, alignment, and the details of hub timing are covered in the *P2 Assembly Language Reference* (Part I) and the *Parallax Propeller 2 Documentation v35 - Rev B/C*.

Pins and smart pins — I/O that thinks for itself

Around the outside of the chip are **64 I/O pins**, numbered P0 through P63. Any cog can read or drive any pin, so pins aren't owned by a particular processor — they're a shared resource, and you decide by convention which cog looks after which pin.

What makes the P2's pins special is that each one is a **smart pin**: a small, self-contained piece of hardware built into the pin itself. You configure a smart pin for a job — measure a pulse, count edges, output a PWM signal, run a serial protocol, do analog-to-digital or digital-to-analog conversion — and then it just *does that job on its own*, without your cog babysitting it. Your cog sets it up once, and afterward only steps in to hand it new data or read back a result.

This is a genuinely different way to do I/O. On most microcontrollers, holding a serial line or measuring a signal precisely means a core has to stay busy doing it. On the P2 you push that work out to the edge of the chip and free the cog entirely. The guiding habit is: **before you write code to bit-bang a protocol, check whether a smart pin already does it in hardware** — usually one does.

There are many smart-pin modes — enough to cover the common serial, timing, counting, and analog jobs — and rather than list them here, we'll send you to the deep reference. The *P2 I/O & Smart Pins User Guide* walks through every mode with examples; this guide just wants you to know the pins are smart and to reach for them first.

P1 NOTE

New in P2: This one has no P1 analog. The P1 had 32 plain general-purpose pins; the P2 has 64, and every one of them is a smart pin. If you spent P1 projects dedicating a cog to bit-bang a UART or a PWM, that work largely moves into the pin hardware on the P2.

The CORDIC solver — shared math hardware

The P2 has a piece of dedicated math hardware called the **CORDIC solver**, shared by all eight cogs. You hand it a number — or a pair of numbers, or an angle — and it hands back results that

would otherwise cost you a lot of code: full 32-bit multiply and divide, square roots, sines and cosines, vector rotations, logarithms and exponentials.

It's *pipelined*, which means it works like an assembly line: you can feed it a steady stream of problems and it keeps producing answers, so several operations from a cog can be in flight at once. For anything involving real math — signal processing, coordinate geometry, generating waveforms — the CORDIC turns work that would be slow in software into something the hardware just does for you.

Here in the orientation we only need you to know it exists and that it's fast and shared. The complete list of operations and exactly how to invoke them lives in the *Parallax Propeller 2 Documentation v35 - Rev B/C*.

P1 NOTE

New in P2: The P1 had no hardware math engine — it shipped log, antilog, and sine *tables* in ROM and you did the rest in software. The P2 replaces that with the CORDIC solver, so the trigonometry and multiply/divide you used to hand-code are now hardware operations.

The streamer — moving data at full speed

Each cog also has its own **streamer**: a dedicated engine for moving data between the hub and the pins (or the analog converters) at the chip's full clock rate, without the cog having to shuttle each piece by hand. You point it at a block of hub memory and a destination, start it, and it streams — while your cog goes on to do something else.

The streamer is how the P2 generates video — VGA, HDMI, and composite signals all come out of it — and it also handles high-speed audio output, fast data capture, and some specialized signal-analysis tricks. Anything that needs a *lot* of data to move *continuously* and *on time* is a job for the streamer.

As with the smart pins, the point right now is just to know it's there. The *P2 Streamer Programming Guide* is the full reference for setting it up and for the video, audio, and capture modes.

P1 NOTE

New in P2: The P1 generated video with a simpler per-cog video generator (the old WAITVID approach). The P2's streamer is a far more capable, general-purpose data mover — video is just one of the things it does.

Events and interrupts — noticing when something happens

Sometimes a cog needs to react to something: a pin changed, a timer reached a count, the CORDIC finished, the streamer is done. The P2 gives each cog a small **event** system for exactly

this. A cog can watch for a hardware condition and then choose how to respond — check on it when convenient, pause until it happens, or let it trigger an **interrupt** that drops into a handler.

What's worth knowing as a newcomer is that on the P2 events are a *convenience, not a necessity*. Because each cog runs its own program independently, you often don't need interrupts at all — you can dedicate a cog to a job and let it watch in a tight loop, with perfectly predictable timing. Events and interrupts are there for when they genuinely simplify a design, not because the chip forces them on you.

P1 NOTE

New in P2: The P1 had no interrupts at all — it used the dedicate-a-cog, poll-in-a-loop model exclusively, and that model still works beautifully on the P2. The P2 *adds* a real event-and-interrupt system per cog as an option for when you want it.

The full set of event sources and how interrupts dispatch are documented in the *P2 Assembly Language Reference* (Part I) and the *Parallax Propeller 2 Documentation v35 - Rev B/C*.

The clock — one setting, the whole chip

All eight cogs and the hardware around them run from a single system clock, and you choose its speed. The P2 has two built-in internal oscillators for when you don't need anything special — a fast one (around 20 MHz) that it starts up on, and a very slow, low-power one — and for real work you attach a crystal and let the P2's on-chip PLL multiply it up to the speed you want, comfortably into the hundreds of megahertz. You set this once, near the top of your program, and the whole chip runs from it.

The practical thing to remember: clock setup is a one-time decision you make up front, not something you fiddle with as you go. Once the chip is running at your chosen speed, every cog's deterministic timing is measured against that one clock.

P1 NOTE

Changed in P2: Same idea as the P1 — one system clock for the whole chip, set up front — but with far more range. Where the P1 topped out at 80 MHz, the P2 runs many times faster, well into the hundreds of megahertz.

Booting — how a program starts running

When the P2 powers up, a small program baked into its **ROM** takes over for the first few milliseconds. It looks at a few designated pins to decide where your program should come from — a serial connection from a host, an SPI flash chip, or a microSD card — loads it, and hands control to Cog 0. That ROM also carries a couple of handy extras: a built-in monitor for poking at a running chip, and even a small Forth interpreter.

For everyday work you mostly don't think about this — your development tools handle loading — but it's good to know the path exists and that the pins used for booting become ordinary I/O once your program is up. The *Parallax Propeller 2 Documentation v35 - Rev B/C* and the boot documentation cover the boot sources and their fallback behavior in detail.

Where this leaves us

That's the whole cast: eight independent cogs, three tiers of memory tied together by a take-turns hub, 64 smart pins doing I/O on their own, a shared CORDIC for math, a per-cog streamer for high-speed data, an event system for reacting to the world, one clock to set, and a ROM that boots you. You don't need to remember every detail — you just need the picture. With it in hand, the next chapter makes sure you can *read* a P2 program — the handful of structural rules that turn Spin2 and PASM2 from a wall of symbols into something legible — and the chapter after that puts these parts to work: we'll launch a cog, drive a pin, and see how a real P2 program is actually shaped.

Chapter 2: Reading P2 Code

Chapter 1 gave you the chip. Before we put it to work, let's make sure you can *read* a P2 program — because the chapters ahead are full of small examples, and they'll only teach you anything if the code on the page isn't a mystery. If you've never seen Spin2 or PASM2, this chapter is your Rosetta stone: by the end you'll be able to read every example in this guide. We are deliberately *not* teaching the whole language here — the reference manuals do that, in depth — we're teaching its **shape**: the small set of structural rules that, once you know them, make P2 code legible. That's a much smaller thing to learn, and it's enough to follow along everywhere else.

One orienting fact first: P2 code is written in *two* languages, and you'll see both. **Spin2** is the high-level language — readable, object-based, where most of a program lives. **PASM2** is the assembly language — the P2's native instructions, used for the small, time-critical pieces. Most of what you'll read is Spin2, so we start there; we meet PASM2 near the end of the chapter. You do not need to *write* either one to read this guide — you need to recognize their parts.

A program is an object — six kinds of blocks

Here's the first and most useful thing to know: **a Spin2 file is an object**, and an object is built from just six kinds of *blocks*. Each block begins with a keyword in the far-left column, and runs until the next such keyword appears. Here is one of each, in a single skeleton — don't read it for what it *does* yet, just for its *shape*:

```
CON                                ' constants: clock, pins, fixed values
  _clkfreq = 200_000_000
  LED      = 56

OBJ                                ' other objects (files) this one builds on
  serial : "jm_full duplexserial"

VAR                                ' this object's own variables
  LONG   count

PUB main()                         ' PUBLIC method; first PUB runs
  count := 0

PRI helper(x) : result             ' a PRIVATE method — an internal helper
  result := x + 1

DAT                                ' data, tables, and PASM2 code live here
  greeting BYTE "hi", 0
```

At the top level, every P2 file is just an arrangement of six blocks:

- **CON — constants.** Fixed values with names: the clock speed, pin numbers, sizes. A file starts in CON mode even before you write the word, so constants can sit at the very top.

- **OBJ — objects.** The other files this one builds on — drivers, libraries, your own code. More on these below.
- **VAR — variables.** This object's own data. Every separate instance of the object gets its own copy of its VARS.
- **PUB — public methods.** The object's *interface* — the methods other code may CALL. **Every program needs at least one**, and the **first PUB is where execution begins** (the boot ROM from Chapter 1 starts it on Cog 0).
- **PRI — private methods.** Internal helpers, callable only from inside this object.
- **DAT — data.** Tables and fixed data — and, as we'll see, PASM2 code.

You won't always use all six; a small program might be just CON and one PUB. But every P2 file you read is some arrangement of these blocks, so spotting the keywords in the left margin tells you instantly how the file is organized.

One distinction to fix in your mind now, because it trips up nearly everyone at first: an **object and a cog are not the same thing**. An object is a unit of *code* — a file you write and compile. A cog (Chapter 1) is a *processor* that runs code. There's no fixed relationship between the two: the methods of one object might run on a single cog, be spread across several, or share a cog with other work. *What runs where* is a decision you make — and deciding it well, for a whole system, is what the companion *P2 Architect's Guide* is about.

P1 NOTE

Same as P1. If you wrote Spin on the Propeller 1, this is home: the same CON/OBJ/VAR/PUB/PRI/DAT block structure, the same “a file is an object,” the same first-PUB-runs rule. Spin2 adds capabilities inside the blocks, but the skeleton is identical — you can skim this chapter and just note what's new.

Methods — how the work is organized

A **method** is a named piece of code you can call — the P2's word for a function or subroutine. You've already seen the shape in the skeleton; here it is named:

```
PUB blink(pin, count) : ok | i
```

Reading left to right: PUB (public) or PRI (private); the method's **name**; its **parameters** in parentheses (pin, count — the values the caller passes in); after the colon, an optional **return value** (ok); and after the bar, optional **local variables** (i) that exist only while the method runs. Most of those parts are optional — PUB main() is a complete, valid header.

You call a method by naming it: blink(56, 10). And here's a point that saves a beginner real confusion: many things that *look* like built-in keywords are actually just **method calls** — pinhigh (LED), waitms(250), cogspin(...) are all methods the language provides, called exactly the way

you'd call your own. There's no separate category to memorize; if it has a name and parentheses, it's a method call.

Indentation is the structure

This is the rule most likely to trip up a newcomer, so we'll say it plainly: **in Spin2, indentation defines structure.** There are no braces, no `begin/end`. A control statement owns exactly the lines indented beneath it. Look:

```
PUB countdown(n)
  repeat n           ' indented lines = the loop body
    pinhigh(LED)
    waitms(100)
    pinlow(LED)
  pinhigh(LED)       ' un-indented: runs after the loop
```

The two `pin` calls and the `waitms` are inside the loop because they're indented under `repeat`; the final `pinhigh` is *not* indented under it, so it runs once after the loop finishes. The three shapes you'll meet most are `repeat` (loop — forever, a fixed count, or while a condition holds), `if/else` (choose), and `case` (choose among many). In every one, what's controlled is what's indented under it.

Values, names, and a line you'll see broken

A few small things, and you'll have seen every kind of token the examples use:

- **`:=` assigns; `=` defines a constant.** Inside a method, `count := 0` puts the value 0 into the variable `count`. Inside `CON`, `LED = 56` names the constant 56. Different jobs — assign with `:=`, name with `=`.
- **Named constants over bare numbers.** We write `LED` once in `CON` and use the name everywhere, rather than scattering 56 through the code. You'll see this habit throughout the guide.
- **Number forms.** Underscores group digits for readability (`200_000_000`); `$` marks hexadecimal (`$1FF`); `%` marks binary (`%1101`). The underscores are purely cosmetic.
- **`@name` means “the address of” name.** Most of the time you pass a variable's *value*; sometimes a method needs to reach the variable (or buffer) itself, and you hand it the address with `@` — you'll see `@stack`, `@count`, `@"some text"` in the examples ahead.
- **A comment** starts with a single quote `'` and runs to the end of the line.
- **`...` continues a line.** A line that ends in three dots `...` continues onto the next, as if the break weren't there — and the rest of the line after the `...` is ignored, so a comment can sit there. We lean on this in the guide to keep a long statement within the page margin, so you'll see it in examples:

```
x := first_term + second_term + third_term ...   ' continued below
    + fourth_term
```

When you see a trailing ..., just read on to the next line as one statement.

Objects — building from other files

Back to OBJ, because composing objects is how real P2 programs are built. An OBJ block pulls in *another* Spin2 file and gives it a name; you then call that file's public methods through the name:

```
OBJ
  serial : "jm_fullduplexserial"    ' the driver file, as "serial"

PUB main()
  serial.start(63, 62, 115_200)    ' call a method through the name
  serial.str(@"hello")
```

`serial` is an *instance* of the driver object; `serial.start(...)` calls the `start` method inside it. This is exactly how you use the community's drivers and your own code: each object minds its own data and exposes methods, and a top-level object wires several together. (The full object model — instances, arrays of objects, parameters — is in the *Spin2 Reference Manual*.)

The other language: PASM2

Everything so far has been Spin2. The P2's *other* language is **PASM2** — its native assembly, the actual instructions the cog runs. You reach for it only where timing has to be exact, but you'll *read* it often, because most drivers have a PASM2 core. So it's worth being able to recognize its shape too.

A line of PASM2 is much flatter than Spin2 — one instruction per line:

```
loop    DRVNOT  #LED           ' toggle the LED pin
        WAITX   ##20_000_000   ' wait ~0.1 s at 200 MHz
        JMP     #loop          ' do it again, forever
```

Read a line left to right: an optional **label** in the left column (`loop` — a name for this spot, so other instructions can jump to it); the **instruction** (`DRVNOT`, `WAITX`, `JMP`); then its **operands**. A `#` before an operand means “this is an immediate value” — a literal number or address, not a register; `##` means a *full 32-bit* immediate (needed for big values like `20_000_000`). Two more parts you'll see but don't need yet: some instructions end in a **flag effect** like `wc` or `wz` (the instruction updates a status flag), and any instruction can carry a **condition** prefix like `if_z` (run only when a flag is set). The deep meaning of all of these is the *P2 Assembly Language Reference*'s job; here, you just need to parse the line.

PASM2 shows up in two places. A whole cog program lives in a **DAT block**; and a short burst can be dropped right inside a Spin2 method between `ORG` and `end`:

```
PUB toggle(pin)
  ORG                ' a little PASM2, inline
  DRVNOT pin
end
```

Either way — and this is the thread back to Chapter 1’s boot story — even a program that’s “all assembly” still lives inside a Spin2 file. Spin2 is always the host.

P1 NOTE

Changed in P2. PASM2 will look familiar to a P1 assembly programmer — the label · instruction · destination, source · effects shape carries straight over. What’s new is *scale*: far more instructions, the `###` full-width immediate, and richer conditionals. If you knew PASM1, you read PASM2 on sight; you’ll just meet new mnemonics.

Where this leaves you

You can now read a P2 program. You know a file is an object built from six kinds of block; that methods are the named pieces of work, and the first `PUB` is where things start; that indentation — not punctuation — is the structure; how values, names, and the `...` continuation look on the page; how objects compose through a name; and what a line of PASM2 is made of. Every example in this guide assumes that much literacy. With it in hand, the next chapter stops reading and starts *doing* — your first real program, a second cog, and the choices a P2 program actually makes.

Chapter 3: Putting It to Work

Now that you can picture the chip *and* read its code, let's use it. This chapter is about *doing* — by the end you'll have driven a pin, launched a second cog, shared data between cogs, and made the one decision every P2 program makes (Spin2 or PASM2?). The point isn't to teach you the whole language — the reference manuals do that, and we'll point you to them — it's to make the chip feel like something you can actually program. We'll keep the examples short, and every one of them compiles.

You met both of the P2's languages in Chapter 2 — Spin2 and PASM2 — so the examples below should read cleanly; this chapter is about putting them to work, not parsing them. Where a program makes a real choice, we'll stop and look at it.

Your first program: drive a pin

Here is a complete, working P2 program. It blinks an LED.

```
CON
    _clkfreq = 200_000_000      ' system clock: 200 MHz
    LED      = 56               ' the pin our LED is on

PUB main()
    repeat                     ' do this forever
        pinhigh(LED)          ' LED on
        waitms(250)           ' wait a quarter second
        pinlow(LED)           ' LED off
        waitms(250)
```

ch03-blink-led.spin2

A few things in that program are worth noticing, because they're true of every P2 program:

- The **CON** block holds **constants**. `_clkfreq` is the one constant you'll almost always set — it tells the chip how fast to run (here, 200 MHz), and everything time-related, like `waitms`, is measured against it. Giving the pin a name (`LED`) instead of scattering the number 56 through your code is the habit we'll keep.
- Execution starts at the **first PUB method** — here, `main`. That's the entry point; the chip runs it on Cog 0 when your program loads.
- `pinhigh`, `pinlow`, and `waitms` are built-in Spin2 methods. Driving a pin really is that direct — name the pin, set it high or low.
- **One board check before you run it.** 56 is the LED pin on a P2 Eval Board and on the standard P2 Edge Module, but the **P2 Edge 32MB Module** puts its two LEDs on **P38 and P39** — and P56 there is a PSRAM clock line, so as written this program would light nothing and write to the memory bus instead. Change `LED` to match your board.

TIP

You don't load this onto the chip by hand — your development tool (PropellerTool, *pnut*, or the VS Code extension) compiles it and sends it over. For now, just read it as “this is what a P2 program looks like.”

P1 NOTE

Changed in P2: Setting the clock is familiar, but simpler and more flexible on the P2: one `_clkfreq` constant near the top of your program, and the compiler works out the PLL settings for you. And pin numbers now run 0–63, not 0–31 — there are twice as many to reach for.

Adding a second Cog

A blinking LED uses one cog and ignores the other seven. The moment that matters is when you give a job to a cog of its own. You do that with `cogspin` — it takes a method to run, hands it to an available cog, and that cog starts running it *alongside* the one you're already on.

```
CON
    _clkfreq = 200_000_000
    LED_A    = 56
    LED_B    = 57

VAR
    LONG stack[64]                ' work space for the second cog

PUB main() | cog
    cog := cogspin(NEWCOG, blink(LED_A, 250), @stack) ' run on another cog
    blink(LED_B, 1000)                ' this cog keeps the slower blink for itself

PRI blink(pin, ms)
    repeat
        pintoggle(pin)
        waitms(ms)
```

ch03-two-cog-blink.spin2

When this runs, **two cogs are blinking at once** — one cog flips LED_A four times a second, the other flips LED_B once a second, and neither one waits on the other. That's the P2's whole personality in five lines: when you want something to happen in parallel, you don't reach for a timer interrupt or a scheduler — you hand the job to a cog and let it run.

Three details that generalize:

- NEWCOG means “any free cog” — you usually don’t care which one. `cogspin` returns the cog number it actually used (or `-1` if all eight were busy).
- That `-1` travels in one direction only. It is what you get *back* on failure, never something you pass *in*. If you meet `COGINIT(-1, ...)` in older or P1-shaped code, it is not asking for “any free cog”: the value arrives as `$FFFF_FFFF`, whose low six bits read as `%111111`, and that starts an even/odd **pair** of cogs — quietly using two where one was meant. `NEWCOG` is the symbol that means “any free one”.
- The new cog needs a little **stack** space in hub to work with; that’s the `LONG stack[64]` we hand it with `@stack` (the `@` means “the address of”).
- `blink` is written once and used by both cogs. A `PUB` method is the public face of your code; a `PRI` method is private to the object. That `PUB/PRI` split is the P2’s run-time model in miniature, which we’ll come back to.

Sharing data between Cogs

Independent cogs still need to talk. The simplest way is the hub: because hub memory is shared, a variable that lives there is visible to every cog. One cog writes it, another reads it — a mailbox.

```
CON
    _clkfreq = 200_000_000
    LED      = 56

VAR
    LONG stack[64]
    LONG count          ' a hub variable - every cog can see it

PUB main()
    cogspin(NEWCOG, ticker(@count), @stack) ' worker updates count in hub
    repeat
        if count & 1          ' read what the worker left for us
            pinhigh(LED)
        else
            pinlow(LED)

PRI ticker(p)
    repeat
        LONG[p]++           ' bump the shared value in hub
        waitms(100)
```

ch03-shared-mailbox.spin2

Here one cog does nothing but increment `count` ten times a second, and the other cog watches `count` and lights the LED on odd values. Neither cog calls the other; they just agree on a spot in hub memory. Single hub reads and writes are *atomic* — a cog always sees a whole value, never half-written — so this simple mailbox is safe. When a hand-off is more than one value, or several

cogs might write at once, the P2 gives you **locks** (16 hardware locks) to guard the exchange. The *P2 Assembly Language Reference* covers the coordination patterns in depth.

P1 NOTE

Changed in P2: Sharing through hub variables works just as it did on the P1, and locks are still how you guard a multi-step exchange. What changed underneath is the egg-beater hub access from Chapter 1: hand-offs and block transfers move faster and stay just as predictable.

Spin2 or PASM2? A decision, not a syntax tour

Every P2 program makes one architectural choice, sometimes many times: should this piece of work be written in **Spin2** or **PASM2**? It helps to see it as a spectrum of three options, not a binary.

- **Spin2** is the high-level language: objects, methods, expressions, quick to write and to read. It runs as interpreted bytecode, so it's slower than assembly, but your program can be large because the bytecodes live in the roomy hub. Reach for Spin2 for application logic, coordination, setup, and anything not on a tight timing budget.
- **PASM2** is native assembly: it runs with the deterministic, cycle-exact timing from Chapter 1 — most register-to-register instructions execute in just two clocks, while branches and hub accesses take more. Reach for a dedicated PASM2 cog when a job must be fast and precise — a video driver, a bit-banged protocol, a tight control loop.
- **Inline PASM2** sits between them: a short burst of assembly dropped right inside a Spin2 method, for when you need native speed for a moment without dedicating a whole cog to it.

That middle option looks like this — the same toggle, but done with one native instruction:

```
CON
  _clkfreq = 200_000_000
  LED      = 56

PUB main()
  repeat
    toggleFast(LED)
    waitms(250)

PRI toggleFast(pin)
  ORG
    DRVNOT pin          ' one native instruction, full speed
  end
```

ch03-inline-pasm-toggle.spin2

The `ORG ... end` block is real PASM2 running inside a Spin2 method. You don't need to read assembly to take the point: the P2 lets you stay in comfortable Spin2 for most of a program and drop to the metal exactly where it pays off.

The honest guidance is the one most experienced P2 developers converge on: **write the application in Spin2, and give the time-critical jobs their own PASM2 cogs**. A typical P2 program uses both, and that's not a compromise — it's the intended shape. For the full languages, the *Spin2 Reference Manual* and the *P2 Assembly Language Reference* are the deep references; this guide only wants you to know *which* tool fits *which* job.

Objects and the run-time model

A P2 program is built from **objects**. An object is a file with its own constants, variables, and methods; you pull one in with an `OBJ` block and `CALL` its methods through a name:

```
OBJ
  serial : "jm_fullduplexserial"    ' a driver object, by filename

PUB main()
  serial.start(63, 62, 115_200)    ' call one of its methods
```

That's how you use the community's drivers and your own: each object minds its own data and exposes methods, and your top-level object wires them together. It's the same mental model as the `PUB/PRI` split you've already seen — public methods are the object's interface, private ones are its internals — scaled up to whole files. The *Spin2 Reference Manual* is the place for the full object model (instances, arrays of objects, parameter passing).

How it boots and runs

You've now seen the run model without us naming it. When the P2 powers up, the boot ROM from Chapter 1 loads your program and starts your **first PUB method on Cog 0**. From there, *your* code is in charge: Cog 0 runs your top object's `main`, and it launches whatever other cogs the design needs with `cogspin` (for Spin2) or its assembly cousin `COGINIT` (for a dedicated PASM2 cog). There's no operating system underneath deciding what runs — the cogs you start are the cogs that run, exactly as you arranged them.

In short, the run model is this: boot loads you, Cog 0 starts your `main`, and your program spreads itself across as many of the eight cogs as it needs.

Where this leaves us

You can now read and shape a real P2 program: set the clock, drive a pin, launch a cog, share data through hub, choose Spin2 or PASM2 for a given job, and compose objects. That is genuinely

enough to build things. What it doesn't yet tell you is *how to decide what goes on which cog* in the first place — how to look at a whole problem and carve it into the right set of cooperating pieces. That decision is where the P2 rewards a little real thought, and it's what the companion *P2 Architect's Guide* is about.

Where to Next

This guide is the orientation layer; the reference manuals are where you go for depth. Here is the map.

- **To write the high-level language** — the *Spin2 Reference Manual* (current revision v55): the full object model, every built-in method and operator, the language's syntax in complete detail.
- **To write assembly** — the *P2 Assembly Language Reference*: the PASM2 instruction set, the execution pipeline, cog start/stop, and the inter-cog coordination primitives (locks, atomic access, cog attention). For a gentler, tutorial-style on-ramp to PASM2, the *DeSilva PASM2 Tutorial* teaches the assembly language from the ground up.
- **For I/O** — the *P2 I/O & Smart Pins User Guide*: every smart-pin mode, with examples — your first stop whenever a protocol might be absorbable at the pin.
- **For high-speed data** — the *P2 Streamer Programming Guide*: the streamer in full, including the video (VGA, HDMI, composite), audio, and capture modes.
- **For debugging and bring-up** — the *P2 Debug Window Manual* and the *P2 Single-Step Debugger Manual*: the on-chip DEBUG output windows and the single-step debugger.
- **For the silicon itself** — the *Parallax Propeller 2 Documentation v35 - Rev B/C*: the foundational reference — CORDIC operations, the event system, boot sources, and the hardware-timing details the other manuals build on.
- **To design a real system on the P2** — *The P2 Architect's Guide*: how to take an application from its peripherals and pin budget through a sound functional decomposition onto cogs, smart pins, and the rest of the fabric. This guide gets you fluent in the P2; the Architect's Guide is the companion that teaches you to *design* with it.

That is the library. Start where your current job points you, and let the picture from Chapter 1, the language from Chapter 2, and the working shape from Chapter 3 guide how you put the pieces together.